

Interroger une IA depuis RStudio

Guide essentiel — Atelier IA & Bioscripting, 12 juin 2026

5 blocs à exécuter dans l'ordre • Ctrl+Enter pour exécuter une ligne

Utilisation d'un LLM depuis R via une API

Nous fournissons un script R prêt à l'emploi permettant d'interroger un modèle de langage (LLM) depuis RStudio au moyen d'une interface de programmation applicative (API).

<https://ifb-elixirfr.github.io/AI-bioscripting-workshop/>

Le script réalise les étapes suivantes :

- Chargement des bibliothèques R nécessaires (**httr2**, **jsonlite**, **base64enc**)
- Configuration de la clé d'accès personnelle à l'API OpenRouter
- Sélection du modèle d'IA à utiliser (GPT-4o, Claude, Mistral, Gemini, Llama, etc.)
- Définition de fonctions R permettant d'envoyer des requêtes textuelles ou des images au modèle
- Réception et affichage des réponses dans la console R

Les diapositives suivantes présentent brièvement chacun de ces éléments avant leur utilisation dans les exercices pratiques.

```
# Atelier IA & Bioscripting 2026 - Guide essentiel ask_ai()
# 12 juin 2026
# Copiez et exécutez chaque bloc avec Ctrl+Enter (Cmd+Enter sur Mac)
# =====

# --- BLOC 1 : INITIALISATION ---
# À exécuter UNE SEULE FOIS en début de session

library(httr2)
library(jsonlite)
library(base64enc)

# Remplacer "VOTRE_CLE_ICI" par l'identifiant de la clé API
# disponible sur le canal TCHAP "labioscripting2-TP-participants-BA"
Sys.setenv(OPENROUTER_API_KEY = "VOTRE_CLE_ICI")

# Choisissez votre modèle - décommentez celui de votre groupe
# MODELE_CHOISI <- "anthropic/claude-sonnet-4-5" # Claude (Anthropic)
MODELE_CHOISI <- "openai/gpt-4o" # GPT-4o (OpenAI)
# MODELE_CHOISI <- "mistralai/mistral-large" # Mistral Large
# MODELE_CHOISI <- "google/gemini-pro-1.5" # Gemini Pro 1.5
# MODELE_CHOISI <- "meta-llama/llama-3.3-70b-instruct" # Llama 3.3 (gratuit)

# Fonction pour questions texte
ask_ai <- function(prompt, model = MODELE_CHOISI, max_tokens = 2000) {
  api_key <- Sys.getenv("OPENROUTER_API_KEY")
  response <- request("https://openrouter.ai/api/v1/chat/completions") |>
    req_headers("Authorization" = paste("Bearer", api_key),
               "content-type" = "application/json") |>
    req_body_json(list(model = model, max_tokens = max_tokens,
                      messages = list(list(role = "user", content = prompt)))) |>
    req_perform()
  result <- response |> resp_body_json()
  cat(result$choices[[1]]$message$content)
  invisible(result$choices[[1]]$message$content)
}

# Fonction pour questions avec image (Claude, GPT-4o, Gemini uniquement)
# Pour simplifier, placez la figure dans votre répertoire de travail RStudio
ask_ai_vision <- function(prompt, image_path, model = MODELE_CHOISI, max_tokens = 2000) {
  api_key <- Sys.getenv("OPENROUTER_API_KEY")
  img_b64 <- base64encode(image_path)
  response <- request("https://openrouter.ai/api/v1/chat/completions") |>
    req_headers("Authorization" = paste("Bearer", api_key),
               "content-type" = "application/json") |>
    req_body_json(list(model = model, max_tokens = max_tokens,
                      messages = list(list(role = "user", content = list(
                        list(type = "image_url",
                             image_url = list(url = paste0("data:image/png;base64,", img_b64))),
                        list(type = "text", text = prompt)))))) |>
    req_perform()
  result <- response |> resp_body_json()
  cat(result$choices[[1]]$message$content)
  invisible(result$choices[[1]]$message$content)
}

# --- BLOC 2 : TEST DE CONNEXION ---
# Exécutez ce bloc avant tout - ne passez à la suite que si ça répond

# Vous pouvez poser toutes les questions sous ce format :
ask_ai("Dis-moi 'Hello world' en français.")

# NB: si vous observez une réponse tranquille :
ask_ai("Votre question", max_tokens = 4000)

# Pour garder la réponse dans une variable :
reponse <- ask_ai("Votre question")
```

BLOC 1 — Initialisation (une seule fois en début de session)

1. Charger les packages

```
library(httr2)
library(jsonlite)
library(base64enc)
```

2. Entrer la clé API (canal TCHAP : iabioscripting2-TP-participants-BA)

```
Sys.setenv(OPENROUTER_API_KEY = "CLE")
```

3. Choisir le modèle (décommenter 1 seule ligne)

```
MODELE_CHOISI <-  
"anthropic/claude-sonnet-4-5"  
# MODELE_CHOISI <- "openai/gpt-4o"  
# MODELE_CHOISI <- "mistralai/mistral-large"  
# MODELE_CHOISI <- "google/gemini-pro-1.5"  
# MODELE_CHOISI <-  
"meta-llama/llama-3.3-70b-instruct"
```

4. Exécuter les fonctions ask_ai() et ask_ai_vision() → voir le fichier .R

BLOC 2 — Test de connexion

Exécutez cette ligne — ne continuez que si vous obtenez une réponse :

```
ask_ai("Dis bonjour en français.")
```

✓ Résultat attendu : "Bonjour ! Comment puis-je vous aider ?"

En cas d'erreur :

Timeout SSL

Attendez 1-2 min et relancez

HTTP 401

Clé API incorrecte — vérifiez la clé TCHAP

req_perform() introuvable

Relancez library(httr2)

Texte vert dans RStudio

Guillemets typographiques — retapez-les

BLOCS 3 & 4 — Questions texte + Questions sur la figure

Question texte

```
ask_ai("Votre question")

# Réponse tronquée ?
ask_ai("Votre question",
      max_tokens = 4000)

# Garder la réponse :
rep <- ask_ai("question")
```

Question sur la figure

```
# Vérifier que la figure est là :
file.exists(
  "fig2A_Scerevisiae
  _heatmap_cible.png")

ask_ai_vision(
  prompt = "Votre question",
  image_path = "fig2A_...png"
)
```

*La figure doit être dans votre répertoire de travail Rstudio
Claude, GPT-4o, Gemini uniquement (pas Llama ni Mistral)*

BLOC 5 — Question avec les données du fichier

```
table1 <- readLines("cell-cycle_SCERE_iabioscripting2.txt", n = 3)
ask_ai(paste0("Voici les 3 premières lignes :\n", paste(table1, collapse="\n"), "\n\nQuel est le
séparateur ?"))
```

Récapitulatif

1

Initialisation

```
libraries + clé TCHAP + modèle + fonctions
```

2

Test de connexion

```
ask_ai("Dis bonjour en français.")
```

3

Question texte

```
ask_ai("Votre question")
```

4

Question sur figure

```
ask_ai_vision(prompt, image_path)
```

5

Question avec données

```
readLines() → injecter dans ask_ai()
```